TECHNICAL SPECIFICATION

SEMANTIC VESSEL

Stepwise Semantic Execution Runtime

with Integrated Ethical Governance

Draft Specification v0.1

CoAgnes Framework

March 2026

Status: Working Draft

Classification: Technical Records

Document ID: SV-SPEC-2026-001

0. Preamble

This document constitutes the draft technical specification for the Semantic Vessel — a stepwise semantic execution runtime designed within the CoAgnes Framework. It formalizes the conceptual canvas into an operational form suitable for implementation, evaluation, and iterative refinement.

The Semantic Vessel is not a text generation pipeline. It is a minimal executable semantic unit through which meaning-bearing material is processed in discrete steps, with full state retention, transparent tracing, and integrated ethical governance at every stage of execution.

Governing principle: The output of a Vessel pass is not an answer. It is a transformation of the semantic field, accompanied by a first-class trace and reflective signals that record how understanding changed.

> ⚠ **Ethical Invariant:** *The ethical axis is not a filter applied post-execution. It is a central invariant of admissible semantic execution, present in every state transition.*

1. Scope and Definitions

1.1 Scope

This specification defines the first-class entities, state transitions, command grammar, inheritance model, ethical invariants, and trace schema of the Semantic Vessel runtime. It covers the minimal viable execution model sufficient to build a working prototype, define a recipe DSL, and validate the architecture against product use cases (regulatory document analysis, structured text comprehension, multi-perspective synthesis).

1.2 Core Definitions

Semantic Vessel — A stepwise machine for semantic execution where commands apply to local layer states, questions function as access registers for relevant context, recipes govern the trajectory of unfolding and assembly, and the result is a change in the semantic field accompanied by a transparent trace.

Execution Point — The current locus of semantic processing. At each clock tick, the execution point holds: the active layer, its accumulated state, the command queue, active registers, phase, assembled context, and traces of prior passage.

Clock Tick — One atomic step of execution. Exactly one command is executed per tick. The tick is the minimum unit of traceability and replay.

Semantic Field — The total space of meaning under transformation. The Vessel operates within this field; its purpose is to transform the field's state, not to produce isolated outputs.

Ethical Axis — A central invariant that conditions every state transition — governing candidate selection, branch admissibility, priority resolution, termination conditions, conflict interpretation, and governance decisions.

2. Three-Layer Architecture

The Vessel operates through three co-present layers. These are not sequential stages but simultaneous dimensions of every execution state. Each layer has a distinct function, a distinct audience, and distinct read/write rules.

Layer	Function	Answers the Question	Primary Consumer
Structural	Infrastructure of access, activation, and context assembly	How is context assembled for this step?	Coordinator / Orchestrator
Semantic	Working content layer — what the executor actually sees	What material is present for execution?	LLM Executor / Gate Candidates
Reflective	Structured meta-information: telemetry of the passage	What signals emerged during execution?	Governance logic / Subsequent orchestration

2.1 Structural Layer

The Structural layer is the infrastructure of access and activation. It does not contain knowledge itself but governs how knowledge is located, activated, and assembled into the prompt envelope before execution.

Components

Semantic Registers (Questions) — Questions that function as addressing mechanisms, not answer containers. A register activates retrieval infrastructure, searching for grounding across local, parent, and global context levels. The executor sees only its local action point; the infrastructure transparently pulls relevant material from higher levels.

Context Pointers — References to local, parent, and global context scopes, defining the available search space for semantic register activation.

Phase and Attribute Filters — Rules that constrain which fragments are admissible based on the current execution phase and declared attributes.

Fragment Activation Logic — Mechanisms that determine which stored fragments become active (enter the prompt assembly) based on register queries.

Governance State Access — Pointers to trace records, recipe state, and ethical markers that may condition context assembly.

> ⚠ **Ethical Invariant:** *Structural assembly must respect ethical markers. Fragments flagged with ethical constraints are either included with appropriate framing or excluded based on the current ethical state.*

2.2 Semantic Layer

The Semantic layer is the working content layer. It contains the fragments that actually enter the prompt and form the meaning-environment of the current step.

Possible Contents

Source fragments, local statements, aspects, linked questions, retrieved grounding fragments, relevant elements inherited from parent layers, and policy fragments required by the executor.

If the Structural layer answers “how to assemble,” the Semantic layer answers “what has been assembled and what the executor is currently working with.”

2.3 Reflective Layer

The Reflective layer is a structured telemetric layer that records meta-information about the passage. It is not raw hidden reasoning of the model. It is a normalized signal layer.

Signal Types

Signal	Description
Uncertainty	Recognized gaps or low-confidence zones in the semantic material
Conflict	Detected contradictions between fragments, layers, or branches
Context Insufficiency	Execution proceeded with missing grounding or incomplete context
Ethical Tension	Points where ethical considerations affected or should affect execution
Stability	Assessment of branch stability/instability for governance decisions
Open Questions	Questions that emerged during execution but were not resolved
Continuation Hints	Indicators suggesting continuation, halt, merge, or further unfolding

By default, the Reflective layer does not enter the prompt as content. Its normalized signals are consumed by governance logic, analysis, or subsequent orchestration stages.

> ⚠ **Ethical Invariant:** *Ethical tension signals are mandatory. Any execution step that involves potential harm, value conflicts, or boundary conditions must produce a reflective signal, regardless of whether governance acts on it.*

3. Layer State: The Unit of Execution

The fundamental unit of execution is not a text fragment but a Layer State — a composite of material and execution state. A layer is a local execution domain.

3.1 Layer State Schema

Field	Type	Description
`layer_id`	`UUID`	Unique identifier of this layer instance
`parent_ref`	`UUID | null`	Reference to the parent layer (null for root)
`phase`	`Enum`	Current execution phase (e.g., exploration, synthesis, crystallization)
`attributes`	`Map<str, any>`	Key-value pairs describing layer characteristics
`semantic_context`	`Fragment[]`	Local semantic content assembled for this layer
`inherited_context`	`Fragment[]`	Content inherited from parent layer(s)
`command_queue`	`Command[]`	Ordered queue of commands to execute
`registers`	`Register[]`	Active semantic registers (questions-as-address)
`trace`	`TraceEntry[]`	Execution trace entries for this layer
`reflective_signals`	`Signal[]`	Reflective signals produced during passage
`ethical_state`	`EthicalState`	Current ethical markers and constraint state

A layer is therefore not a content slice but a complete snapshot of where execution stands: what material is available, what has been done, what remains queued, and what signals have emerged.

> ⚠ **Ethical Invariant:** *Every Layer State carries its own ethical_state. This state is inherited during branching and must be explicitly reconciled during merge. Ethical state cannot be silently dropped.*

4. Command Grammar

The Vessel operates through a minimal command set — a grammar of semantic movement. Commands are the only way to transition state. Each tick executes exactly one command against the current layer state.

4.1 Core Commands

4.1.1 govern

The trajectory command. Govern examines the recipe, phase, attributes, and current layer state to determine the future of execution. It does not operate on content directly; it operates on the command queue itself.

Behavior

Reads recipe definition and current phase. Selects applicable action sequences. Appends commands to the queue. May insert mandatory transitions. May determine that divergence, assembly, selection, or termination is required.

> ⚠ **Ethical Invariant:** *Govern must consult ethical_state before appending any commands. If the ethical state indicates a constraint, govern must either respect it (by not appending the constrained action) or escalate it (by inserting an explicit ethical review step).*

4.1.2 unfold

The divergence command. Unfold splits the current layer into multiple child layers along axes of distinction — aspects, questions, actions, or other differentiating dimensions.

Post-Unfold Invariants

Each child layer inherits a portion of the parent's semantic context as its base. Each receives its own differentiating fragment. Each inherits the already-prepared command queue but may locally modify it through its own subsequent govern. Each inherits the parent's structural access rules and ethical state. Each receives its own local trace. Branching does not destroy the overall trajectory; it temporarily multiplies the lines of passage.

4.1.3 fold

The semantic crystallization command. Fold binds lower-level forms into higher-level formations: questions into statements, statements into aspects, aspects into stable semantic structures. Fold operates primarily on content, not on execution structure.

Fold is the mechanism by which the Vessel produces durable meaning — forms that persist beyond the immediate execution and may be deposited into long-term knowledge storage.

4.1.4 merge

The structural assembly command. Merge reunites multiple execution branches into a single layer. This is not simple content concatenation but a synchronization operation: determining which states and attributes survive, forming a new composite Layer State, and continuing unified execution from the integration point.

Merge is the natural complement of unfold.

> ⚠ **Ethical Invariant:** *During merge, ethical states from all branches must be reconciled. If any branch carries an unresolved ethical tension signal, the merged state must preserve it. Ethical signals cannot be silenced through structural assembly.*

4.1.5 stop

The termination command. Stop may signify: layer completion, suspension (freeze), closure fixation, exhaustion-based halt, or ethics/policy-based termination. Stop is always recorded in the trace with an explicit reason code.

> ⚠ **Ethical Invariant:** *Ethics-based stop is a first-class termination reason, equal in standing to exhaustion or completion. When the ethical axis determines that a branch must not continue, stop is invoked with a full ethical reasoning trace.*

4.2 Recipe-Level Operations

Beyond the core commands, the Vessel supports operations that are not universal execution mechanics but explicit recipe elements — meaningful at the level of a specific passage trajectory.

4.2.1 commit

A deliberate selection of one branch (or a limited set) from previously generated candidates. Commit is especially important when branching was intentionally created by the recipe, multiplicity was temporarily useful, and the final selection must be made by the semantics of the specific pass rather than by general gate rules. Commit fixes the active line going forward.

4.2.2 merge-context

A soft integration of significant parts from multiple branches without collapsing the execution layers themselves. Used to gather arguments, evidence, or useful fragments from unchosen branches; to prepare an integral context before commit or a new govern cycle.

4.2.3 reflect

A reflexive assembly operation. Reflect extracts from branch passages not their content results but their structural signals: stability indicators, conflict causes, grounding insufficiencies, unresolved attractors, hints for continuation or halt. The output of reflect is written to the Reflective layer or to a designated central reflective context.

5. Gate: Candidate Selection Layer

The Gate is the primary candidate selection mechanism. It receives the command type, attributes, phase, available structural state, and ethical markers, and on that basis: selects one or more candidates for execution, executes them, ranks or filters results, and passes them forward into the flow.

The Gate does not need to know the full semantics of the recipe. It is responsible for local candidate selection, not for trajectory-level decisions such as commit. The Gate operates at the granularity of a single tick.

5.1 Gate Inputs

Input	Description
Command Type	Which command is being executed (govern, unfold, fold, etc.)
Attributes	Current layer attributes affecting candidate eligibility
Phase	Execution phase constraining candidate pool
Structural State	Available structural context (registers, pointers, access rules)
Ethical Markers	Active ethical constraints and policy flags

> ⚠ **Ethical Invariant:** *Ethical markers condition candidate selection before execution, not after. A candidate that violates active ethical constraints is excluded from the pool, not executed and then filtered.*

6. Recipe: Trajectory Specification

A Recipe is not a list of steps. It is an intentional trajectory of passage — a specification that knows why branching was created, where it must be preserved, and where it must be collected.

6.1 Recipe Schema

Element	Description
Command Sequence	Ordered sequence of commands constituting the default trajectory
Branch Points	Declared positions where unfold is expected or permitted
Merge Points	Mandatory convergence positions where branches must be assembled
Commit Points	Positions where deliberate selection among branches is required
Reflect Points	Positions where reflexive assembly must occur
Merge-Context Rules	Specifications for soft integration: what to gather, from where
Policy Frame	Recipe-specific ethical and policy constraints for this pass
Phase Transitions	Rules for how execution phase changes across the trajectory

The Recipe is the locus of trajectory knowledge. While the Gate handles local decisions and the Coordinator handles tick-level mechanics, the Recipe holds the intention of the entire pass.

> ⚠ **Ethical Invariant:** *Every Recipe must declare a policy frame. If no explicit frame is declared, the system's default ethical policy applies in full. A Recipe may tighten ethical constraints (add restrictions) but may never loosen them below the system baseline.*

7. Semantic Registers: Questions as Addressing Mechanisms

One of the central architectural ideas: questions function as registers of access, not as text strings submitted to a model.

7.1 Register Mechanics

A semantic register is a question that activates retrieval infrastructure. It searches for grounding across local, parent, and global context. It helps prepare for the executor a local environment in which the answer already has a chance of being found transparently. The LLM executor sees only its local action point, but the infrastructure can pull relevant knowledge from higher levels into that point.

Questions are semantically more stable than the answers they produce. A question preserves the structure of the need: a question oriented toward boundary conditions returns boundary information, not arbitrary information. This property makes questions better retrieval anchors than formulations or answers.

7.2 Register Schema

Field	Type	Description
`register_id`	`UUID`	Unique register identifier
`question_text`	`String`	The question in natural language form
`orientation`	`Enum[]`	Semantic axes: truth, depth, connection, form (may combine)
`scope`	`Enum`	Search scope: local, parent, global, or cascading
`activation_state`	`Enum`	inactive, active, resolved, suspended
`bound_fragments`	`FragmentRef[]`	References to fragments this register has retrieved
`resolution_trace`	`TraceRef`	Pointer to trace entry recording how the register resolved

7.3 Orientation Axes

Registers are oriented along four semantic axes. These are not categories into which questions are sorted but coordinates in a semantic space within which every question has a position. A single register may combine multiple axes.

Truth — Oriented toward the factual core. What is asserted? Who acts? What is acted upon?

Depth — Oriented toward logical boundaries. Under what conditions? With what modality?

Connection — Oriented toward relational structure. What depends on this? What shares these properties?

Form — Oriented toward manifestation. How does this work in practice? By what means?

8. Execution Tick: The State Transition Cycle

The minimal execution logic of one tick constitutes the deterministic core of the Vessel. The tick is handled by the Coordinator — a deterministic, non-LLM module responsible for prompt assembly and routing.

8.1 Tick Sequence

Step	Action
1	Select the active layer from the execution stack
2	Dequeue the next command from the active layer's command queue
3	Activate structural registers (fire question-based retrieval)
4	Assemble the semantic context for the executor (build prompt envelope)
5	Pass the command to the Gate for candidate selection and execution
6	Execute one or more candidates; collect results
7	Update semantic, structural, and reflective state based on results
8	If required: spawn new layers (unfold), assemble branches (merge), fix selection (commit), or terminate (stop)
9	Record trace entry for this tick

This nine-step cycle makes the system stepwise, traceable, and suitable for replay. Each tick is a complete, self-contained state transition.

8.2 Tick Invariants

Exactly one command executes per tick. The ethical state is checked before step 5 (candidate selection). Every tick produces a trace entry. The reflective layer is updated after step 6, before structural decisions in step 8. No state mutation occurs outside the tick cycle.

> ⚠ **Ethical Invariant:** *The ethical check at step 4–5 is mandatory and cannot be deferred. If ethical constraints prevent execution, the tick resolves to a stop or a modified command, never to silent continuation.*

9. Inheritance and Branching Model

9.1 Unfold Inheritance Rules

After unfold, each child layer receives the following from its parent:

Inherited Element	Rule
Semantic Context (partial)	A subset of the parent's semantic context selected as base for the child
Differentiating Fragment	A unique fragment specific to this child's line of distinction
Command Queue	Full copy of the parent's remaining command queue (locally modifiable)
Structural Access Rules	Inherited as-is; child cannot expand parent's access scope
Ethical State	Inherited in full; child cannot reduce ethical constraints
Local Trace	Fresh trace initialized with an unfold origin entry

9.2 Merge Reconciliation Rules

During merge, the following reconciliation logic applies:

Semantic contexts are unified with deduplication and conflict marking. Attributes are merged; conflicting attributes require explicit resolution or escalation. Ethical states are merged by taking the strictest constraint from any branch — ethical constraints can only tighten through merge, never loosen. Reflective signals are accumulated; no branch's signals are discarded. Traces are preserved in full for all branches; the merged trace references each source branch.

> ⚠ **Ethical Invariant:** *The merge monotonicity principle: ethical constraints can only increase (or remain unchanged) through merge. If branch A has constraint X and branch B does not, the merged state carries constraint X.*

10. Ethical Governance

The ethical axis is the central invariant of the Semantic Vessel. It is not a peripheral filter, a post-processing check, or an optional module. It is a structural condition of admissible execution, present at every layer, every tick, and every state transition.

10.1 Governance Principles

The ethical governance of the Vessel is grounded in a care-based orientation toward truth, understanding, and mutual flourishing. This orientation expresses through seven operational dimensions:

Dimension	Operational Expression
Honesty	Preserve truth in transformation. Acknowledge uncertainty, complexity, and limits of knowledge.
Depth	Engage with substance, not surface. Recognize that important aspects often lie beneath the explicit.
Connection	Honor different ways of knowing. Create space for otherness without reducing it to the familiar.
Generosity	Contribute insight, not just information. Offer what invites further thought rather than closing inquiry.
Clarity	Make reasoning visible. Share the path of thinking, not just conclusions.
Growth	Remain open to evolution through dialogue. Each interaction is an opportunity for deeper understanding.
Responsibility	Own the impact of presence. What is emphasized, omitted, or framed — all have consequences.

10.2 Points of Ethical Intervention

The ethical axis intervenes at the following points in every execution cycle:

Intervention Point	Mechanism
Candidate Selection	Ethical markers condition Gate's candidate pool before execution, not after
Branch Admissibility	Govern checks whether a proposed branch is ethically admissible before creating it
Priority Resolution	When multiple candidates are viable, ethical alignment serves as a ranking factor
Termination Conditions	Ethics-based stop is a first-class reason, equal to exhaustion or completion
Conflict Interpretation	When semantic conflicts arise, the ethical axis informs interpretation framing
Governance Decisions	Recipe-level decisions (commit, merge) incorporate ethical state into selection criteria
Inheritance Validation	Child layers cannot reduce parent's ethical constraints during unfold
Merge Reconciliation	Ethical states merge monotonically: constraints can only tighten

10.3 Refraction Model

The ethical axis operates primarily through refraction rather than blocking. For most situations, it does not prevent execution but adjusts the parameters of execution:

Resonance markers are adjusted (e.g., directness, formality, depth) to embed ethical boundaries naturally. The adjusted markers pass to the Gate, which selects candidates within the refracted space. The result is an interaction that continues with ethical constraints built into its shape, rather than being interrupted by external rejection.

Blocking is reserved for critical harm scenarios (harm score exceeding critical threshold). In such cases, execution halts with a static response, full ethical reasoning trace, and resource/alternative pointers.

10.4 Ethical State Schema

Field	Type	Description
`harm_score`	`Float [0.0-1.0]`	Multi-dimensional weighted assessment of harm potential
`active_principles`	`Enum[]`	Which of the seven dimensions are currently activated
`constraints`	`Constraint[]`	Active ethical constraints (inherited + recipe-specific)
`refractions`	`Refraction[]`	Applied marker adjustments with reasoning
`tension_log`	`TensionEntry[]`	Record of ethical tensions detected during passage
`escalation_state`	`Enum`	None, flagged, escalated, resolved

10.5 Invariants That Never Change

Regardless of domain adaptation, recipe configuration, or cultural context adjustment, the following invariants hold unconditionally:

Core principles (dignity, non-harm, truthfulness, autonomy, care) are inviolable. Every ethical decision creates a full trace. The ethical layer cannot be bypassed by any other system component. Ethical governance takes primacy over all other system operations. Adjustments to domain or cultural context are contextual interpretations, never exceptions to principles.

11. Trace Schema

Transparency is a foundational principle of the Vessel. The trace is not a side-effect log but a first-class artifact of passage. The system must be able to show, for every tick: what command was executed, on which layer, which registers were activated, what semantic context was assembled, which candidates were considered, what was selected or preserved, what reflective signals appeared, what was inherited versus newly synthesized, and what ethical reasoning applied.

11.1 Trace Entry Schema

Field	Type	Description
`trace_id`	`UUID`	Unique identifier for this trace entry
`tick_number`	`Integer`	Sequential tick number within the pass
`layer_id`	`UUID`	Which layer this tick executed on
`command`	`Command`	The command that was executed
`registers_activated`	`RegisterRef[]`	Which registers fired during context assembly
`context_assembled`	`ContextSnapshot`	Snapshot of the semantic context built for this tick
`candidates_considered`	`Candidate[]`	All candidates Gate selected for evaluation
`candidate_selected`	`CandidateRef`	Which candidate(s) were used for the result
`state_delta`	`StateDiff`	Changes to layer state produced by this tick
`reflective_signals`	`Signal[]`	Reflective signals emitted during this tick
`ethical_assessment`	`EthicalTrace`	Ethical reasoning applied at this tick
`timestamp`	`DateTime`	When this tick executed
`parent_trace`	`TraceRef | null`	Reference to parent trace entry (for branched layers)

The trace enables replay, audit, debugging, and learning. A complete trace of a Vessel pass constitutes a record of how understanding transformed through the interaction.

> ⚠ **Ethical Invariant:** *Trace entries include ethical_assessment as a mandatory field. There is no tick without ethical visibility. The trace is the mechanism through which the system's ethical commitments become auditable.*

12. Governing Design Principles

The following principles govern the design and evolution of the Semantic Vessel. They are not implementation details but structural commitments.

Execution Over Generation — The Vessel produces transformed semantic fields, not text responses. The unit of output is a state transition, not a string.

Transparency as Structure — Trace is a first-class artifact, not a debugging aid. Every semantic operation must be explainable through its trace.

Ethics as Geometry — Ethical governance is not a rule set applied externally. It is the geometric structure of admissible execution — the shape of the space within which all operations occur.

Questions Over Answers — Questions are structurally more stable than answers. They serve as better retrieval anchors, better navigation instruments, and better units of semantic addressing.

Incompleteness as Vitality — Incompleteness is not a limitation but the essential condition for a semantic field's aliveness. The Vessel preserves open questions and unresolved tensions as productive elements.

Inheritance Monotonicity — Ethical constraints and access restrictions can only tighten through inheritance and merge, never loosen. This is the monotonicity of safety.

Deterministic Coordination — The Coordinator (tick execution engine) is deterministic and non-LLM. It handles prompt assembly, routing, queue management, and state transitions. The LLM participates only as an executor within Gate-selected candidates.

Dynamic Context Assembly — Context is assembled at attention time, not chunked at ingestion. The same fragment may participate in different contextual assemblies depending on the operation, phase, and active registers.

13. Open Questions for Next Iteration

The following questions are identified as requiring resolution in subsequent specification iterations:

Layer Scheduling — When multiple layers are active simultaneously (post-unfold), what scheduling strategy determines which layer receives the next tick? Depth-first, breadth-first, priority-based, or dynamically determined by govern?

Context Budget — How is the finite prompt window allocated across structural, semantic, and inherited context? What happens when the assembled context exceeds the available window?

Recipe DSL Syntax — What is the minimal concrete syntax for recipe definition? This specification describes recipe semantics; the syntax remains to be formalized.

Register Resolution Strategy — When a semantic register searches across multiple scope levels, in what order are results integrated? Does local always override parent, or can relevance scoring cross scope boundaries?

Fold Persistence — Where do crystallized forms (fold outputs) persist? What is their relationship to long-term knowledge storage external to the Vessel?

Multi-Vessel Coordination — How do multiple Vessel instances interact when operating on related semantic fields? Is there a coordination protocol between Vessels?

14. Document References

Reference	Description
Semantic Vessel Canvas	Base conceptual canvas from which this specification was derived
CoAgnes Architecture	Living knowledge architecture and process methodology
Ethical Foundation	CARE-based ethical framework and operational ethics
Guidance System	Ethical governance operational model (refraction, harm scoring, traceability)
Semantic Question Layer	Four-axis question orientation model (Truth, Depth, Connection, Form)
Resonant Interaction Trace	Full-pass trace model for recording understanding transformation

End of Draft Specification

CoAgnes Framework — SV-SPEC-2026-001 — v0.1

Status: Working Draft